

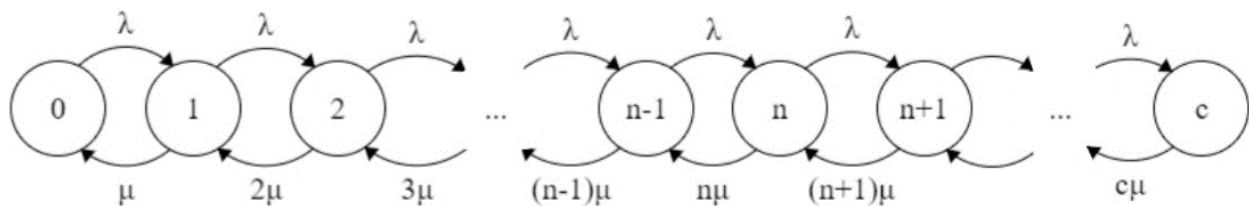
Ναταλία Μπούρδη

03119031

Ανάλυση και Σχεδιασμός τηλεφωνικού κέντρου

Μοντελοποιούμε ένα τηλεφωνικό κέντρο ως μια ουρά M/M/c/c με αφίξεις Poisson ρυθμού λ , και εξυπηρετήσεις εκθετικής κατανομής μέσου ρυθμού μ .

(1) Διάγραμμα ρυθμού μεταβάσεων M/M/c/c με εξυπηρετητές ίδιων δυνατοτήτων:



Θα αποδείξουμε ότι ισχύει ο ακόλουθος τύπος (Erlang-B) για την πιθανότητα απόρριψης πελάτη από το σύστημα:

$$P_{blocking} = B(\rho, c) = \frac{\rho^c / c!}{\sum_{k=0}^c \rho^k / k!}, \text{ όπου } \rho = \frac{\lambda}{\mu} \text{ η ένταση φορτίου.}$$

Οι εργοδικές εξισώσεις κατάστασης:

$$1. P_k = \left(\frac{\lambda}{\mu} \right) P_{k-1} = \left(\frac{\lambda}{k\mu} \right) P_{k-1}, \quad k = 1, \dots, c.$$

$$2. \sum_{k=0}^c P_k = 1$$

Από την εξίσωση 1 έχουμε

$$P_k = \frac{\lambda}{k\mu} \cdot \frac{\lambda}{(k-1)\mu} P_{k-2} = \frac{\lambda^2}{(k-1)k\mu^2} \cdot P_{k-2} = \frac{\lambda^3}{(k-2)(k-1)k\mu^3} \cdot P_{k-3} = \dots \text{ κ.ο.κ. } \Rightarrow$$

$$P_k = \frac{\lambda^k}{k! \mu^k} P_0.$$

Αντικαθιστούμε στην εξίσωση 2, οπότε έχουμε

$$\sum_{k=0}^c \frac{\lambda^k}{k! \mu^k} P_0 = 1 \Rightarrow \sum_{k=0}^c \frac{\rho^k}{k!} P_0 = 1 \Rightarrow P_0 = \frac{1}{\sum_{k=0}^c \frac{\rho^k}{k!}}.$$

Τελικά, καταλήγουμε στον τύπο Erlang-B:

$$P_{blocking} = P_c = \frac{1}{\sum_{k=0}^c \frac{\rho^k}{k!}} \frac{\rho^c}{c!} = \frac{\frac{\rho^c}{c!}}{\sum_{k=0}^c \frac{\rho^k}{k!}}.$$

Ο μέσος ρυθμός απωλειών πελατών, $\lambda - \gamma$, από το σύστημα είναι η πιθανότητα απόρριψης $P_{blocking}$ ενός πελάτη από το σύστημα επί τον ρυθμό άφιξης των πελατών λ . Δηλαδή,

$$\lambda - \gamma = \lambda P_{blocking} = \lambda \cdot \frac{\frac{\rho^c}{c!}}{\sum_{k=0}^c \frac{\rho^k}{k!}}.$$

Με βάση τον τύπο που αποδείξαμε κατασκευάζουμε συνάρτηση `erlangb_factorial`.

```
# Erlang-B factorial

function p_blocking = erlangb_factorial(rho, c)
    sum=0;
    for k=0:1:c
        sum += power(rho, k)/factorial(k);
    endfor
    num = power(rho,c)/factorial(c);
    p_blocking = num/sum;
endfunction
```

Επαληθεύουμε την ορθότητα της συνάρτησής μας με τη βοήθεια της `erlangb` του `queueing` package της Octave, δοκιμάζοντάς την για $\rho = 2$ και $c = 3$.

```
>> display(erlangb(2,3));
    0.21053
>> display(erlangb_factorial(2,3));
    0.21053
```

(2) Χρησιμοποιώντας τώρα τον επαναληπτικό τύπο Erlang-B για τη πιθανότητα blocking

$$B(\rho, 0) = 1$$

$$B(\rho, n) = \frac{\rho B(\rho, n-1)}{\rho B(\rho, n-1) + n}, \quad n = 1, 2, \dots, c$$

κατασκευάζουμε επαναληπτική συνάρτηση `erlangb_iterative`.

```
# Erlang-B iterative

function B = erlangb_iterative(rho, c)
    B = 1;
    for n=0:1:c
        num = rho*B;
        den = rho*B + n;
        B = num/den;
    endfor
```

```
endfor
endfunction
```

Το αποτέλεσμα για $\rho = 2$ και $c = 3$ είναι το ίδιο με της `erlangb_factorial`.

```
>> display(erlangb_iterative(2,3));
0.21053
```

(3) Συγκρίνουμε τις τιμές `erlangb_factorial(1024, 1024)` και `erlang_iterative(1024, 1024)`.

```
>> display(erlangb(1024,1024));
0.024524
>> display(erlangb_iterative(1024,1024));
0.024524
>> display(erlangb_factorial(1024,1024));
NaN
```

Η `iterative` υλοποίηση μας επιστρέφει το αναμενόμενο αποτέλεσμα, όπως επαληθεύεται από την `erlangb` του `queueing` package, ενώ η `factorial` υλοποίηση καταλήγει σε υπερχείλιση και αδυνατεί να μας επιστρέψει αποτέλεσμα λόγω του μεγάλου ορίσματος στις συναρτήσεις `power` και `factorial`.

(4) Για τον σχεδιασμό τηλεφωνικού δικτύου εταιρείας 200 εργαζομένων, μετρήσαμε στην ώρα αιχμής χρήση για εξωτερικές κλήσεις του πιο απαιτητικού χρήστη 23 λεπτά ανά ώρα.

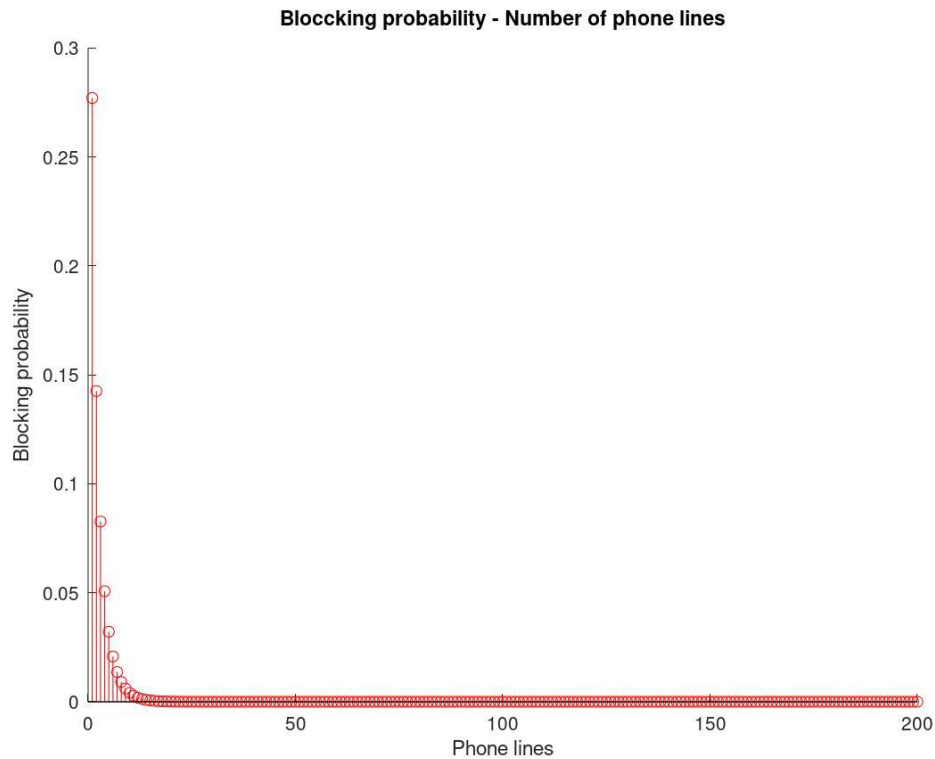
(4α) Σχεδιάζουμε με βάση τον πιο απαιτητικό χρήστη, δηλαδή θεωρώντας ότι όλοι οι εργαζόμενοι είναι το ίδιο απαιτητικοί με αυτόν. Θεωρούμε ότι οι εξωτερικές κλήσεις ακολουθούν διαδικασία Poisson, και οι εξυπηρετήσεις εκθετική με μέση διάρκεια, βάσει του απαιτητικότερου χρήστη, $1/\mu = 23$ min.

Οπότε, η συνολική ένταση φορτίου του συστήματος είναι

$$\rho = \frac{\lambda}{\mu} = \frac{200/60}{1/23} = 76.67 \text{ Erlangs}$$

(4β) Χρησιμοποιούμε τον τύπο Erlang-B μέσω της συνάρτησης που δημιουργήσαμε, `erlangb_iterative`, ώστε να παραγάγουμε το διάγραμμα της πιθανότητας απόρριψης κλήσης ως συνάρτηση του αριθμού των τηλεφωνικών γραμμών.

```
1 # Blocking probabilities diagram
2 # using erlangb_iterative
3 clc;
4 close all;
5
6 for c=1:1:200
7     rho = 23*c/60;
8     pblock(c) = erlangb_iterative(rho, c)
9 endfor
10
11 figure(1);
12 stem(pblock, 'r');
13 title("Blocking probability - Number of phone lines");
14 xlabel("Phone lines");
15 ylabel("Blocking probability");
```



(4γ) Θέλουμε αριθμό τηλεφωνικών γραμμών τέτοιο, ώστε η πιθανότητα blocking κλήσης να είναι $< 1\%$. Χρησιμοποιούμε το παρακάτω πρόγραμμα.

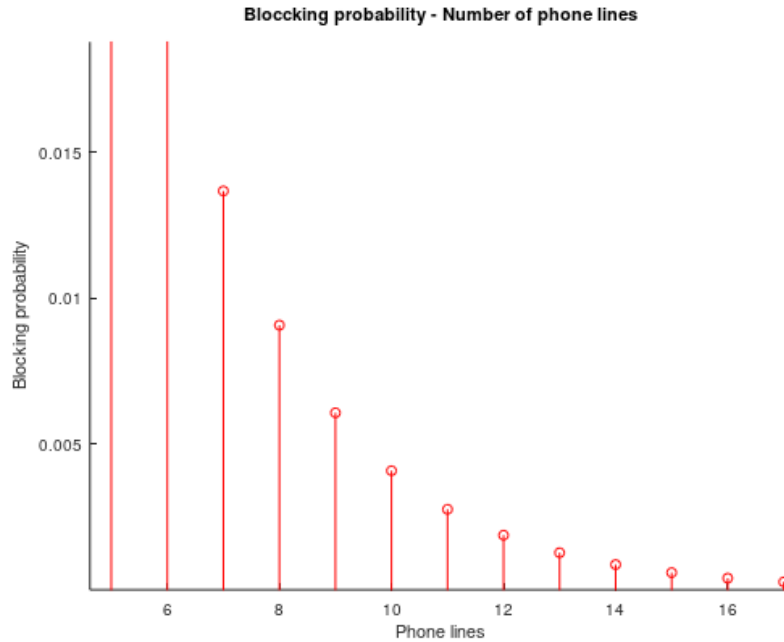
```

1 # pblocking < 1%
2
3 clc;
4 close all;
5
6 for c=1:1:200
7     rho = 23*c/60;
8     pblock = erlangb_iterative(rho, c)
9     if pblock < 0.01
10         display(c);
11         break;
12     endif
13 endfor

```

Προκύπτει ότι για 8 τηλεφωνικές γραμμές ή πιθανότητα απόρριψης γίνεται μικρότερη του 1% και ίση με 0.9%.

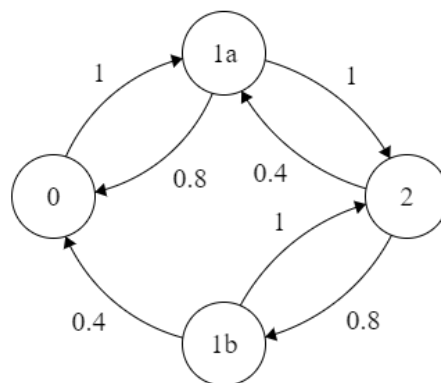
Η ίδια πληροφορία φαίνεται γραφικά από το διάγραμμα του ερωτήματος β.



Σύστημα εξυπηρέτησης με δύο ανόμοιους εξυπηρετητές

Διαθέτουμε σύστημα εξυπηρέτησης χωρίς δυνατότητα αναμονής πελατών που αποτελείται από δύο εξυπηρετητές (1 και 2) με προτεραιότητα στον πρώτο εξυπηρετητή, ο οποίος είναι υψηλότερων δυνατοτήτων. Οι αφίξεις πελατών ακολουθούν κατανομή Poisson με μέσο ρυθμό $\lambda = 1$ πελάτες/sec και οι εξυπηρετήσεις είναι εκθετικά κατανεμημένες με μέσους χρόνους εξυπηρέτησης $\frac{1}{\mu_1} = 1.25\text{sec}$ και $\frac{1}{\mu_2} = 2.5\text{sec}$ για τον εξυπηρετητή 1 και 2 αντίστοιχα ($\mu_1 = 0.8$ πελάτες/sec και $\mu_2 = 0.4$ πελάτες/sec).

(1) Το διάγραμμα ρυθμών μεταβάσεων του συστήματος στην κατάσταση ισορροπίας:



Διάγραμμα ρυθμών μεταβάσεων

(α) Οι εργοδικές πιθανότητες του συστήματος:

$$\lambda P_0 = \mu_1 P_{1a} + \mu_2 P_{1b} \Rightarrow P_0 = 0.8P_{1a} + 0.4P_{1b}$$

$$(\lambda + \mu_1)P_{1a} = \lambda P_0 + \mu_2 P_2 \Rightarrow 1.8P_{1a} = P_0 + 0.4P_2$$

$$(\lambda + \mu_2)P_{1b} = \mu_1 P_2 \Rightarrow 1.4P_{1b} = 0.8P_2$$

$$P_0 + P_{1a} + P_{1b} + P_2 = 1$$

Λύνοντας το σύστημα εξισώσεων καταλήγουμε στις πιθανότητες:

- $P_0 = 0.2495$
- $P_{1a} = 0.2144$
- $P_{1b} = 0.1949$
- $P_2 = 0.3411$

(β) Η πιθανότητα απόρριψης πελάτη από το σύστημα [P(blocking)] ισούται με την πιθανότητα το σύστημα να βρεθεί στην κατάσταση 2.

$$P_{blocking} = P_2 = 0.3411$$

(γ) Ο μέσος αριθμός πελατών στο σύστημα $E[N(t)]$.

$$E[N(t)] = \sum_{k=0}^2 kP_k = P_{1a} + P_{1b} + 2P_2 = 0.2144 + 0.1949 + 2 \cdot 0.3411 = 1.0915$$

(2) Προσομοιώνουμε το παραπάνω σύστημα στο Octave.

(α) Όταν βρισκόμαστε στην κατάσταση 0, μεταβαίνουμε στην κατάσταση 1 (1a) ανεξαρτήτως του τυχαίου αριθμού που παράγεται.

Όταν βρισκόμαστε στην κατάσταση 1 (1a) και ο τυχαίος αριθμός που παράγεται είναι μικρότερος από το `threshold_1a`, θεωρούμε άφιξη στον εξυπηρετητή 1b, αλλιώς αναχώρηση από τον εξυπηρετητή 1a. Επομένως, `threshold_1a = lambda/(lambda+m1)`

Όταν βρισκόμαστε στην κατάσταση 2 (1b στο διάγραμμα μεταβάσεων) και ο τυχαίος αριθμός είναι μικρότερος από το `threshold_1b`, θεωρούμε ότι υπήρξε άφιξη, αλλιώς αναχώρηση από τον εξυπηρετητή 1b. Οπότε, `threshold_1b = lambda/(lambda+m2)`

Όταν βρισκόμαστε στην κατάσταση 3 (2 στο διάγραμμα μεταβάσεων) και ο τυχαίος αριθμός που παράγεται είναι μικρότερος από το `threshold_2_first`, θεωρούμε ότι υπήρξε άφιξη (και επομένως απόρριψη), αλλιώς αν είναι μικρότερος από το `threshold_2_second` αναχώρηση και μετάβαση στην 1b με μ_1 , αλλιώς αναχώρηση και μετάβαση στην 1a με μ_2 .

Θέτουμε `threshold_2_first = lambda/(lambda+m1+m2)` και `threshold_2_second = (lambda+m1)/(lambda+m1+m2)`

(β) Ως κριτήριο σύγκλισης, ελέγχουμε ανά 1000 επαναλήψεις τη διαφορά του μέσου αριθμού πελατών που υπολογίσαμε στην τρέχουσα επανάληψη με αυτόν που υπολογίστηκε στην προηγούμενη επανάληψη. Όταν

η διαφορά αυτή γίνει μικρότερη του 0.001%, θεωρούμε τη προσομοίωση ικανοποιητική και τερματίζουμε το πρόγραμμα.

(γ) Οι εργοδικές πιθανότητες P_0 έως P_2 που υπολογίζει το πρόγραμμα:

```
Command Window
0.24299
0.21500
0.19425
0.34776
```

Παρατηρούμε ότι είναι περίπου ίσες με αυτές που υπολογίσαμε θεωρητικά. Διαφέρουν λίγο λόγω της προσεγγιστικής φύσης της προσομοίωσης και του κριτηρίου σύγκλισης.

Ο κώδικας:

```
clc;
clear all;
close all;

lambda = 1;
m1 = 0.8;
m2 = 0.4;

threshold_1a = lambda/(lambda+m1);
threshold_1b = lambda/(lambda+m2);
threshold_2_first = lambda/(lambda+m1+m2);
threshold_2_second = (lambda+m1)/(lambda+m1+m2);

current_state = 0;
arrivals = zeros(1,4);
total_arrivals = 0;
maximum_state_capacity = 2;
previous_mean_clients = 0;
delay_counter = 0;
time = 0;

while 1 > 0
    time = time + 1;

    if mod(time,1000) == 0
        for i=1:1:4
            P(i) = arrivals(i)/total_arrivals;
        endfor

        delay_counter = delay_counter + 1;

        mean_clients = 0*P(1) + 1*P(2) + 1*P(3) + 2*P(4);

        delay_table(delay_counter) = mean_clients;

        if abs(mean_clients - previous_mean_clients) < 0.00001
```

```

        break;
    endif
    previous_mean_clients = mean_clients;
endif

random_number = rand(1);

if current_state == 0
    current_state = 1;
    arrivals(1) = arrivals(1) + 1;
    total_arrivals = total_arrivals + 1;
elseif current_state == 1
    if random_number < threshold_1a
        current_state = 3;
        arrivals(2) = arrivals(2) + 1;
        total_arrivals = total_arrivals + 1;
    else
        current_state = 0;
    endif
elseif current_state == 2
    if random_number < threshold_1b
        current_state = 3;
        arrivals(3) = arrivals(3) + 1;
        total_arrivals = total_arrivals + 1;
    else
        current_state = 0;
    endif
else
    if random_number < threshold_2_first
        arrivals(4) = arrivals(4) + 1;
        total_arrivals = total_arrivals + 1;
    elseif random_number < threshold_2_second
        current_state = 2;
    else
        current_state = 1;
    endif
endif

endwhile

display(P(1));
display(P(2));
display(P(3));
display(P(4));

```